



程序设计原理

字符数组

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



CStrings

Strings are used often in programming. You have used string literals. A string is a sequence of characters. There are two ways to process strings in C++. One way is to process strings using the string class. The other way is to treat strings as arrays of characters. This is known as pointer-based strings or C-strings.



Initializing Character Arrays

➤ Two ways to declare C-string Using an array variable

```
char city[7] = "Dallas";  
char city[] = {'D', 'a', 'l', 'l', 'a', 's', '\0'};  
char city[] = "Dallas";  
cout << city;
```

char city[7]; city= "Dallas";



'D'	'a'	'l'	'l'	'a'	's'	'\0'
city[0]	city[1]	city[2]	city[3]	city[4]	city[5]	city[6]

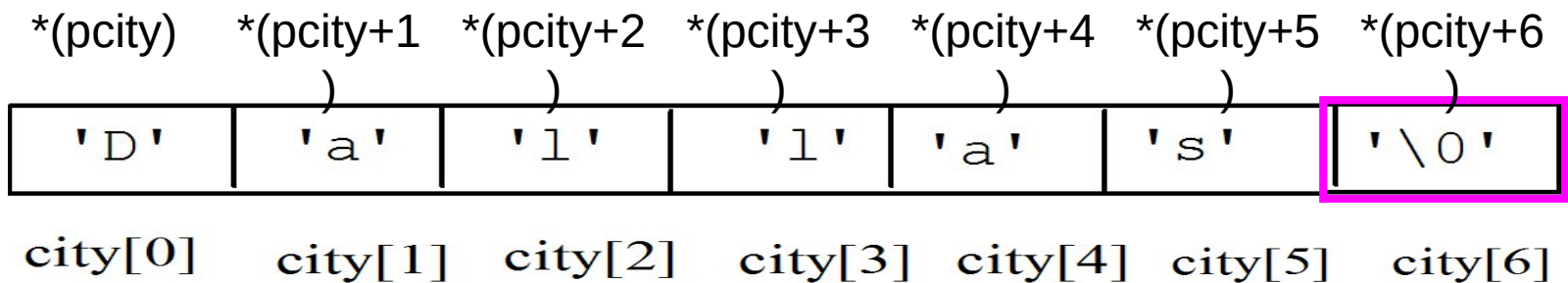


C-string using Pointer

- Declare a string variable using a pointer

```
const char *pCity = "Dallas";
```

```
cout<<pCity;
```





Example

```
#include <iostream>
using namespace std;
int main()
{ char str1[ ]="character array";
  const char *ps="character point";
  cout<<str1<<endl;
  cout<<ps<<endl;
  return 0;
}
```

character array
character point



Example

```
#include <iostream>
using namespace std;
int main()
{ char s1[20]="abcde",*ps1;
  char s2[ ]="xyzf";
  int i=0;
  ps1=s1;
  while (*ps1) ps1++; //寻找s1字符串的结束位置
  int j=0;
  while(*(s2+j)){ //将s2字符串逐个复制到s1后
    *ps1=*(s2+j);
    ps1++;
    j++;
  }
  *ps1='\0';
  cout<<s1<<endl;
  return 0;
}
```

程序执行结果：
abcdexyzf



Reading Strings

- Using “cin”: input stops at the first whitespace (space, tab, newline). It cannot read strings containing spaces.

```
char city[10];  
cin >> city;
```

```
char *city2;  
cin >> city2;
```



No memory is allocated to store the content of city2.

- Using “getline” : can read any character.

- a function defined in iostream

`cin.getline(char array[], int size, char delimiterChar)`

Where to store?

The last character is reserved for `\0`.

Where to stop?

delimiterChar is read but not stored.
Default delimiterChar is `'\n'`.



Input and Output of C-Strings

To output a C-Strings is simple. Suppose s is an array for C-Strings. To display it to the console:

```
cout << s;
```

You can read a string from the keyboard using the cin object. For example, see the following code:

```
cout << "Enter a city: ";  
cin >> city; // read to array city  
cout << "You entered " << city << endl;
```




Character Test Functions

Function	Description	Example
<code>isdigit(c)</code>	Returns true if <code>c</code> is a digit.	<code>isdigit('7')</code> is true <code>isdigit('a')</code> is false
<code>isalpha(c)</code>	Returns true if <code>c</code> is a letter.	<code>isalpha('7')</code> is false <code>isalpha('a')</code> is true
<code>isalnum(c)</code>	Returns true if <code>c</code> is a letter or a digit.	<code>isalnum('7')</code> is true <code>isalnum('a')</code> is true
<code>islower(c)</code>	Returns true if <code>c</code> is a lowercase letter.	<code>islower('7')</code> is false <code>islower('a')</code> is true
<code>isupper(c)</code>	Returns true if <code>c</code> is an uppercase letter.	<code>isupper('a')</code> is false <code>isupper('A')</code> is true
<code>isspace(c)</code>	Returns true if <code>c</code> is a whitespace character.	<code>isspace('\t')</code> is true <code>isspace('A')</code> is false
<code>isprint(c)</code>	Returns true if <code>c</code> is a printable character including space ' '.	<code>isprint(' ')</code> is true <code>isprint('A')</code> is true
<code>isgraph(c)</code>	Returns true if <code>c</code> is a printable character excluding space ' '.	<code>isgraph(' ')</code> is false <code>isgraph('A')</code> is true
<code>ispunct(c)</code>	Returns true if <code>c</code> is a printable character other than a digit, letter, or space.	<code>ispunct('*')</code> is true <code>ispunct(',')</code> is true <code>ispunct('A')</code> is false
<code>iscntrl(c)</code>	Returns true if <code>c</code> is a control character such as <code>'\n'</code> , <code>'\f'</code> , <code>'\v'</code> , <code>'\a'</code> , and <code>'\b'</code> .	<code>iscntrl('*')</code> is false <code>iscntrl('\n')</code> is true <code>iscntrl('\f')</code> is true



Case Conversion Functions

Function	Description	Example
<code>tolower(c)</code>	Returns the lowercase equivalent of <code>c</code> , if <code>c</code> is an uppercase letter. Otherwise, return <code>c</code> itself.	<code>tolower('A')</code> returns <code>'a'</code> <code>tolower('a')</code> returns <code>'a'</code> <code>tolower('\t')</code> returns <code>'\t'</code>
<code>toupper(c)</code>	Returns the uppercase equivalent of <code>c</code> , if <code>c</code> is a lowercase letter. Otherwise, return <code>c</code> itself.	<code>toupper('A')</code> returns <code>'A'</code> <code>toupper('a')</code> returns <code>'A'</code> <code>toupper('\t')</code> returns <code>'\t'</code>



String Functions

Function	Description
<code>int strlen(char *s1)</code>	Returns the length of the string, i.e., the number of the characters before the null terminator.
<code>char *strcpy(char *s1, const char *s2)</code>	Copies the string s2 to string s1. The value in s1 is returned.
<code>char *strncpy(char *s1, const char *s2, size_t n)</code>	Copies at most n characters from string s2 to string s1. The value in s1 is returned.
<code>char *strcat(char *s1, const char *s2)</code>	Appends string s2 to s1. The first character of s2 overwrites the null terminator in s1. The value in s1 is returned.
<code>char *strncat(char *s1, const char *s2, size_t n)</code>	Appends at most n characters from string s2 to s1. The first character of s2 overwrites the null terminator in s1 and appends a null terminator to the result. The value in s1 is returned.
<code>int *strcmp(char *s1, const char *s2)</code>	Returns a value greater than 0, 0, or less than 0 if s1 is greater than, equal to, or less than s2 based on the numeric code of the characters.
<code>int *strncmp(char *s1, const char *s2, size_t n)</code>	Returns a value greater than 0, 0, or less than 0 if the n characters in s1 is greater than, equal to, or less than the first n characters in s2 based on the numeric code of the characters.
<code>int atoi(char *s1)</code>	Converts the string to an int value.
<code>double atof(char *s1)</code>	Converts the string to a double value.
<code>long atol(char *s1)</code>	Converts the string to a long value.
<code>void itoa(int value, char *s1, int radix)</code>	Converts the value to a string based on specified radix.



Exercise

Shuffle

Your task is to shuffle a deck of n cards, each of which is marked by a alphabetical letter.

A single shuffle action takes out h cards from the bottom of the deck and moves them to the top of the deck.

The deck of cards is represented by a string as follows.

abcdeefab

The first character and the last character correspond to the card located at the bottom of the deck and the card on the top of the deck respectively.

For example, a shuffle with $h = 4$ to the above deck, moves the first 4 characters "abcd" to the end of the remaining characters "eefab", and generates the following deck:

eefababcd

You can repeat such shuffle operations. Write a program which reads a deck (a string) and a sequence of h , and prints the final state (a string).



Exercise

Input

The input consists of multiple datasets.
Each dataset is given in the following format:

A string which represents a deck

The number of shuffle m

h1

h2

.

.

hm

The input ends with a single character '-' for the string.

Constraints

The length of the string ≤ 200

$1 \leq m \leq 100$

$1 \leq h_i < \text{The length of the string}$

The number of datasets ≤ 10

Output

For each dataset, print a string which represents the final state in a line.

Sample Input

aabc

3

1

2

1

vwxyz

2

3

4

-

Sample Output

aabc

xyzvw



Exercise

Transformation

Write a program which performs a sequence of commands to a given string `str`. The command is one of:

print a b: print from the a-th character to the b-th character of `str`

reverse a b: reverse from the a-th character to the b-th character of `str`

replace a b p: replace from the a-th character to the b-th character of `str` with `p`

Note that the indices of `str` start with 0.

Input

In the first line, a string `str` is given. `Str` consists of lowercase letters. In the second line, the number of commands `q` is given. In the next `q` lines, each command is given in the above mentioned format.

Output

For each print command, print a string in a line.



Exercise

Constraints

$1 \leq \text{length of str} \leq 1000$

$1 \leq q \leq 100$

$0 \leq a \leq b < \text{length of str}$

for replace command, $b - a + 1 = \text{length of p}$

Sample Input 1

abcde

3

replace 1 3 xyz

reverse 0 2

print 1 4

Sample Output 1

xaze

Sample Input 2

xyz

3

print 0 2

replace 0 2 abc

print 0 2

Sample Output 2

xyz

abc